



ESCUELA DE INGENIERÍA
FACULTAD DE INGENIERÍA

EDUCACIÓN
PROFESIONAL

Olvido Catastrófico y Aprendizaje Continuo

Un problema real en el uso
continuo de modelos

Profesor:

Alain Raymond





Agenda de Hoy

1

Motivación del Problema

2

Aprendizaje Incremental

3

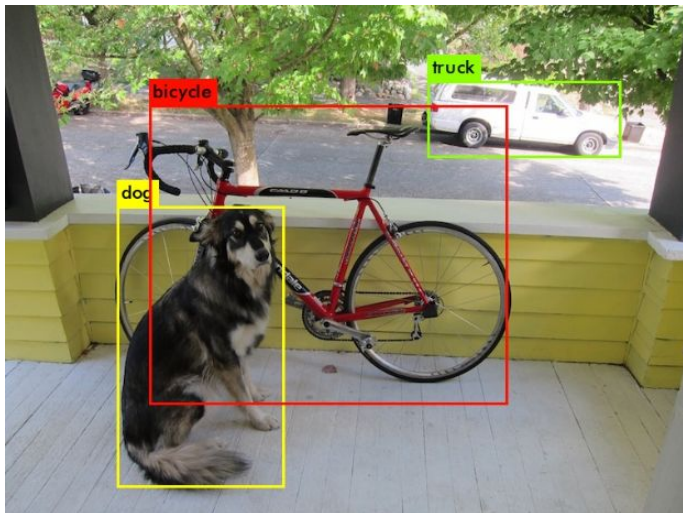
Escenarios de Aprendizaje
Incremental

4

Métodos para lidiar con el Olvido
Catastrófico

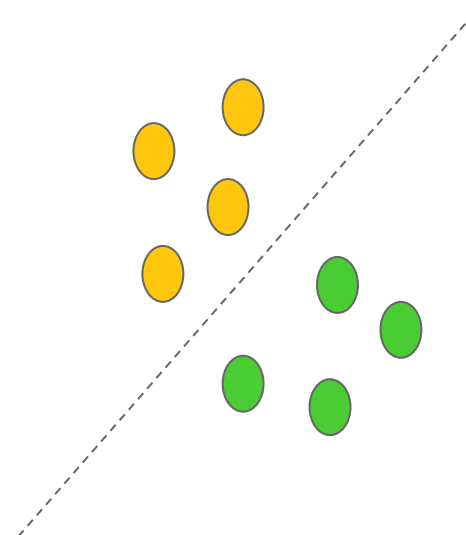


Motivación





Motivación



Motivación

- Datos de una nueva clase
- Datos de un contexto o dominio diferente
- Los datos de las clases actuales cambian





Motivación

¿Qué podemos hacer cuando
llegan nuevos datos?

Entrenar con
todos los datos

Entrenar sólo con
los **nuevos datos**

Entrenar **nuevos**
modelos

Todos los casos tienen Ventajas, Desventajas y
Problemas



Entrenar con
todos los datos

Entrenar solo
con los nuevos
datos

Entrenar nuevos
modelos

Espacio en
disco



Transferencia



Tiempo
entrenamiento



Acceso a Datos



Olvido

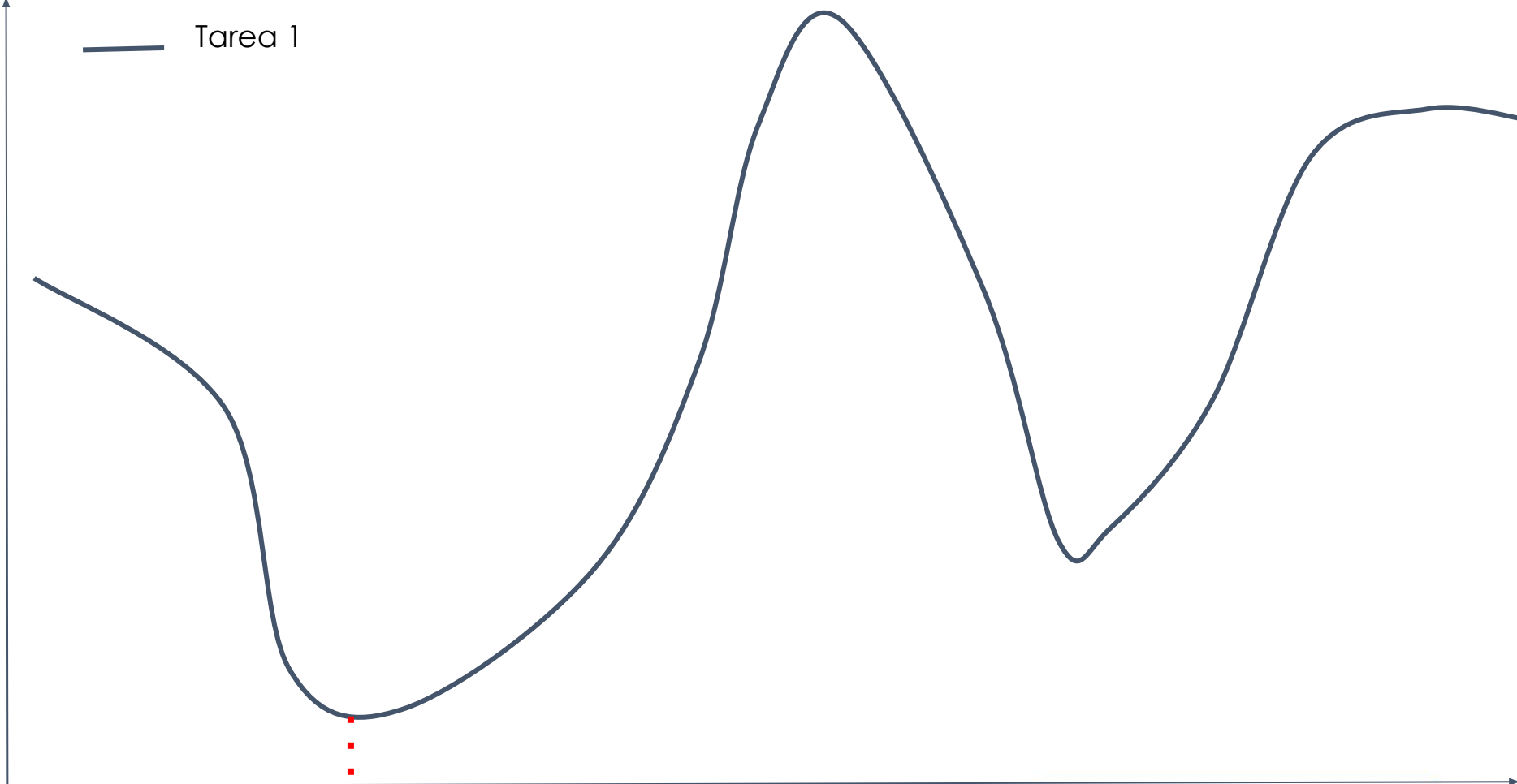




¿Qué significa olvidar?

Función Pérdida

Tarea 1

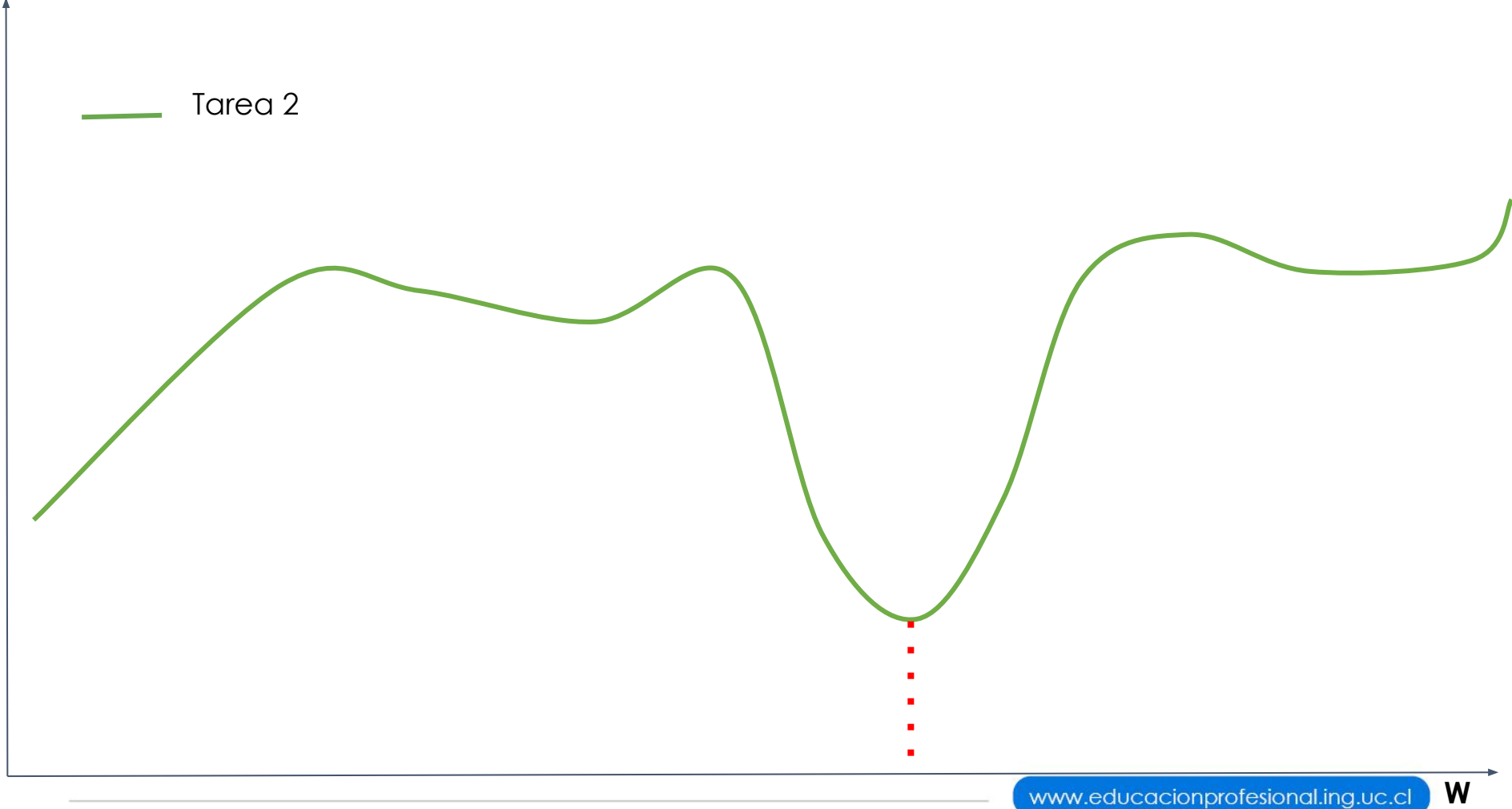




¿Qué significa olvidar?

Pérdida

Tarea 2



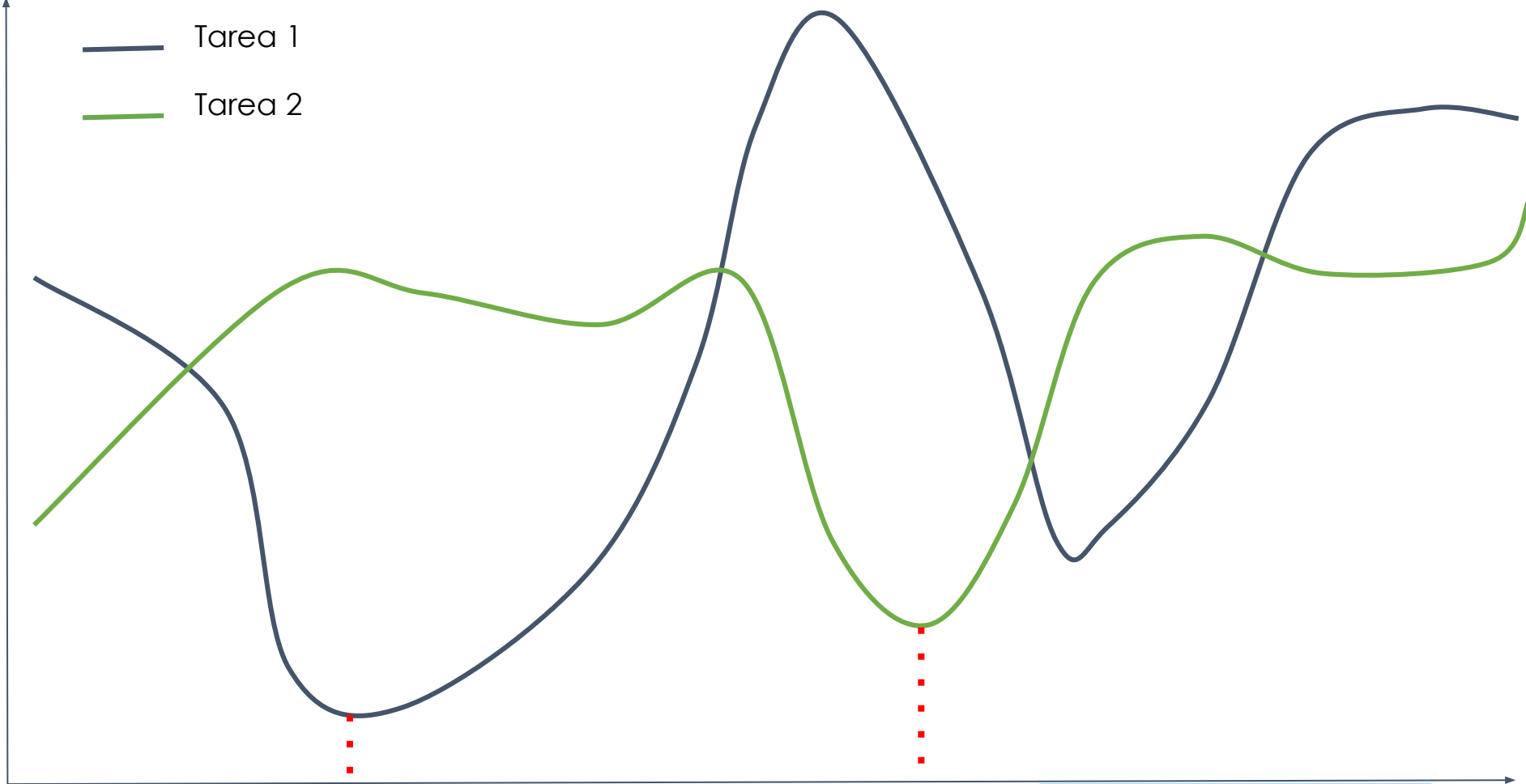


¿Qué significa olvidar?

Pérdida

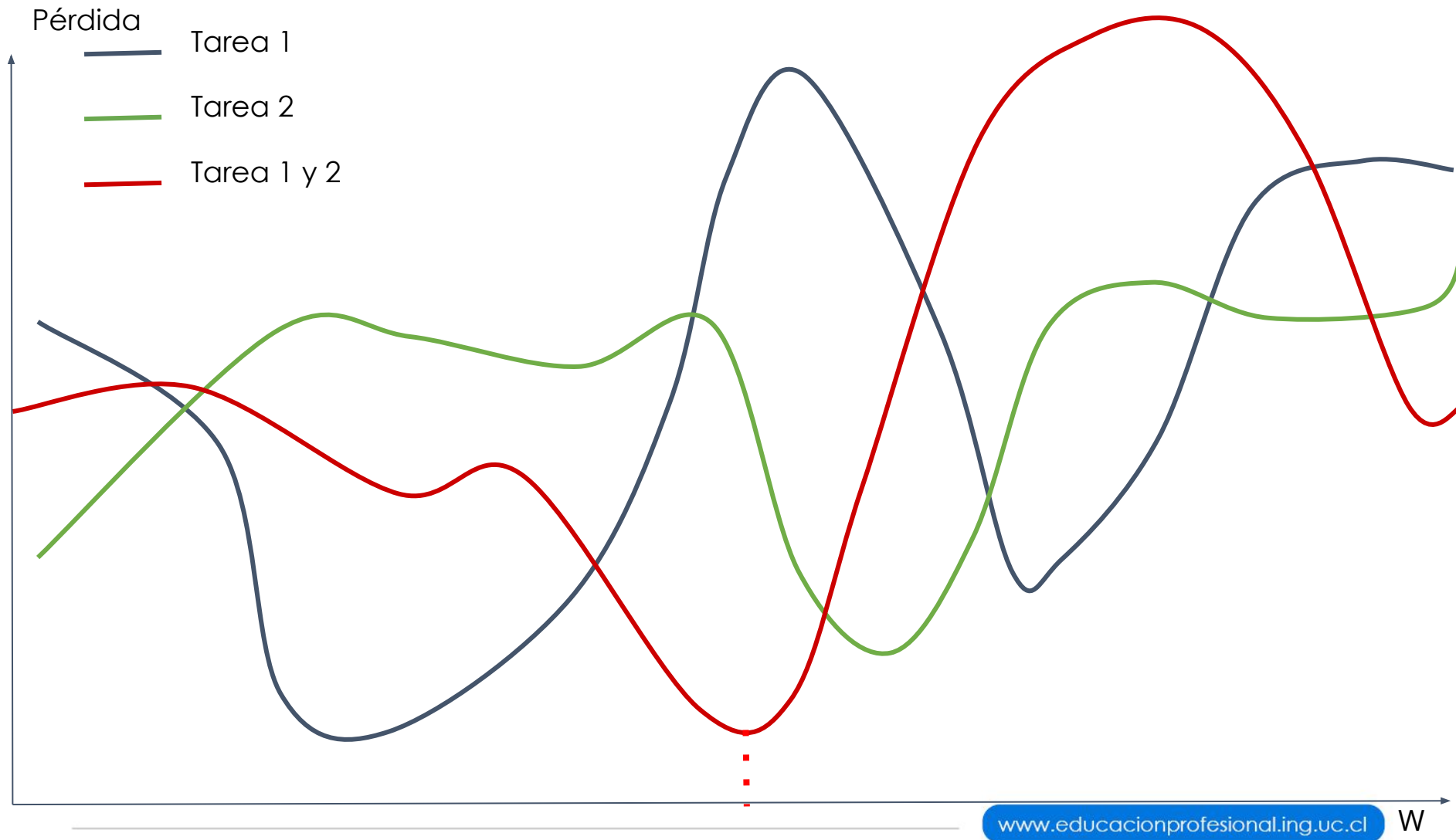
Tarea 1

Tarea 2



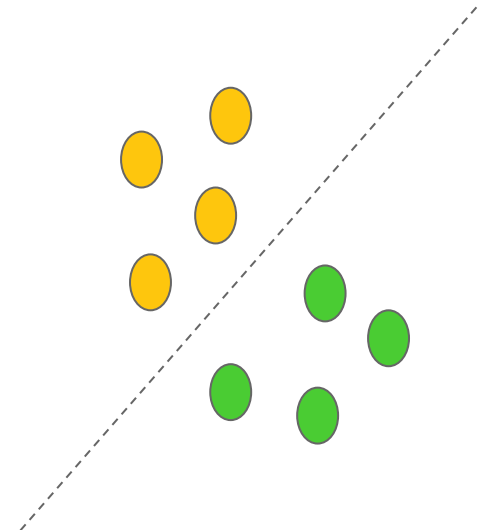


¿Qué significa olvidar?



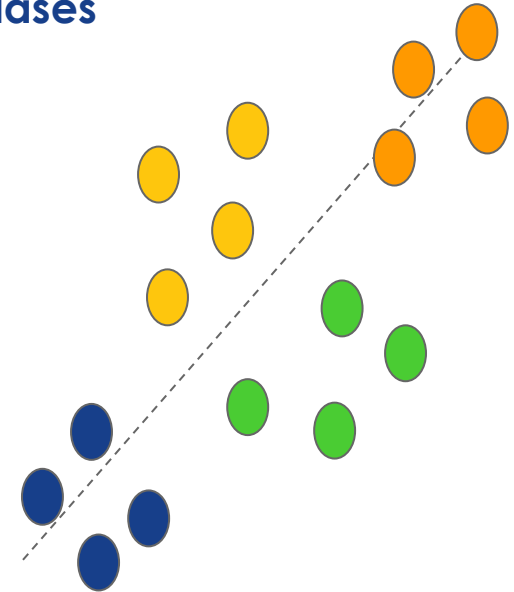
Aprendizaje Incremental

- Entrenamos con los datos disponibles (Tarea 1)
- Optimizamos con los datos de la tarea disponible



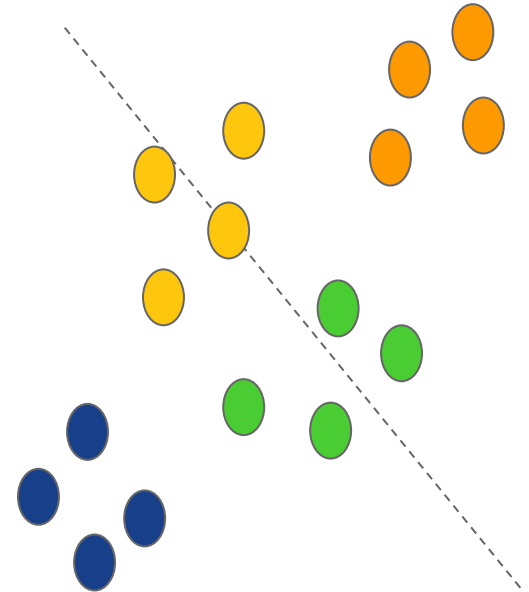
Aprendizaje Incremental

- Llegan nuevos datos (Tarea 2)
- El modelo no está preparado para estas nuevas clases



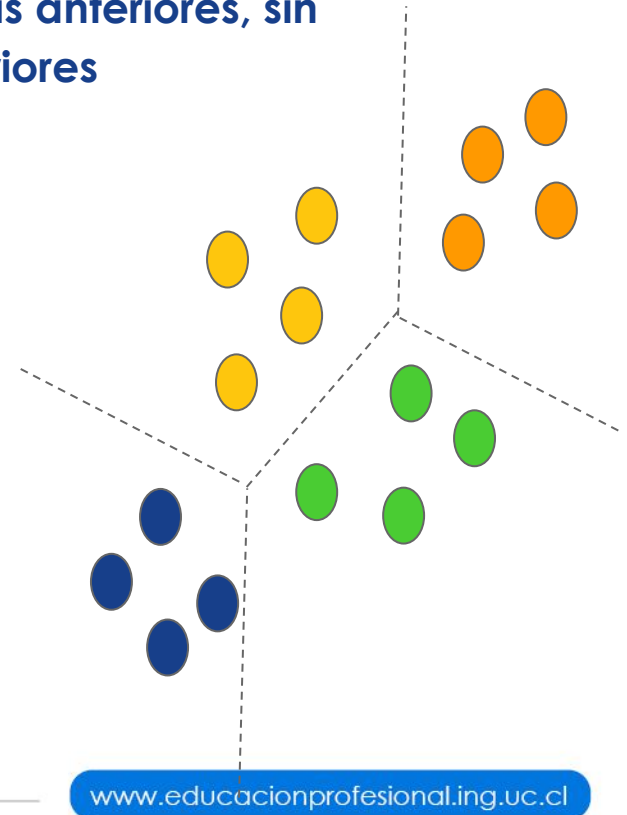
Aprendizaje Incremental - Problema

- Entrenamos con los datos disponibles (Tarea 2)
- Optimizando solo para la tarea 2, pierde rendimiento en la tarea anterior



Aprendizaje Incremental - Ideal

- Entrenamos con los datos disponibles (Tarea 2)
- Optimizando los pesos considerando tareas anteriores, sin requerir datos anteriores



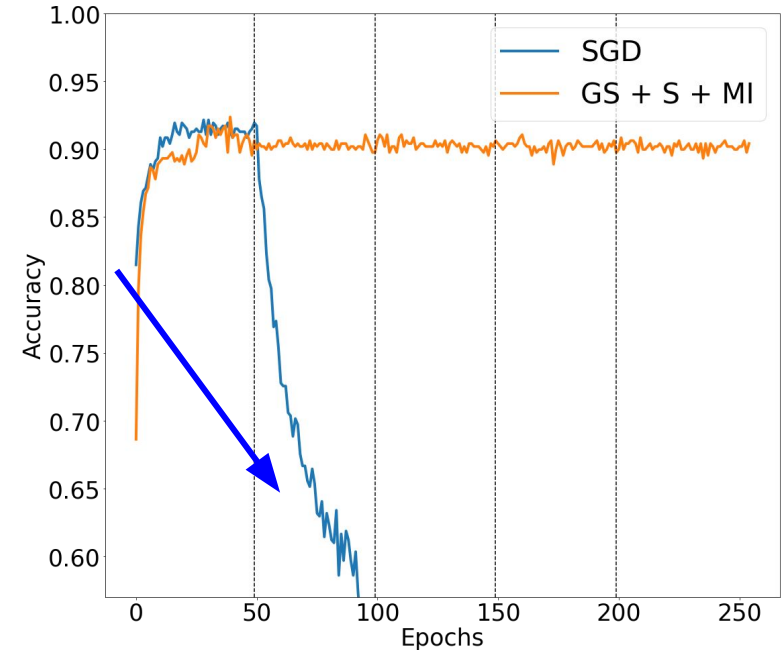
Aprendizaje Incremental - Problema

- **Problema:**

- *Olvido Catastrófico*
- El modelo olvida lo aprendido en el pasado

- **Objetivo:**

- Buscamos que el modelo mantenga su rendimiento en tareas en las que previamente fue entrenado.





Olvido Catastrófico - Causas

- **El modelo optimiza sus pesos pensando sólo en la tarea actual**
 - Olvida lo aprendido en el pasado para aprender la nueva tarea
- **La modificación de los pesos causa *interferencia* entre tareas**

El olvido es el síntoma del problema en Continual Learning



Aprendizaje Incremental

- **Objetivos:**

- Aprender la nueva tarea
- Retener información de tareas anteriores
 - Eliminar o disminuir la interferencia
- Mejorar el rendimiento de tareas pasadas y futuras
 - Transferencia Positiva

- **Con restricciones:**

- Sólo con datos de la tarea actual
- Memoria en disco
- Identificador de tareas
- Otras

¿Cómo definimos una tarea?



C_1

C_2

D^1



C_3

C_4

D^2



C_5

C_6

D^3

.....



Aprendizaje Incremental

- Dependiendo de las restricciones del problema, se pueden definir 3 escenarios:

Task Incremental

Class Incremental

**Domain
Adaptation**

Task Incremental

- La tarea es un nuevo dataset
 - Con una cantidad de clases independientes
- Cada tarea tiene su propio identificador

Tarea 1



C 1

C 2

Tarea 2



C 1

C 2

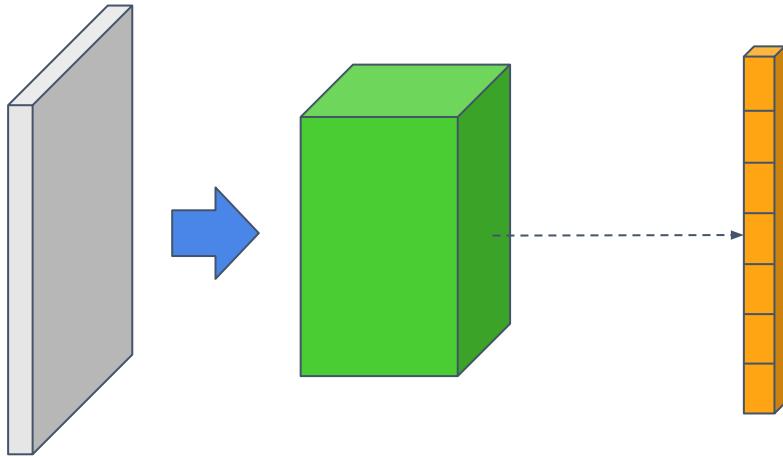
Tarea 3



C 1

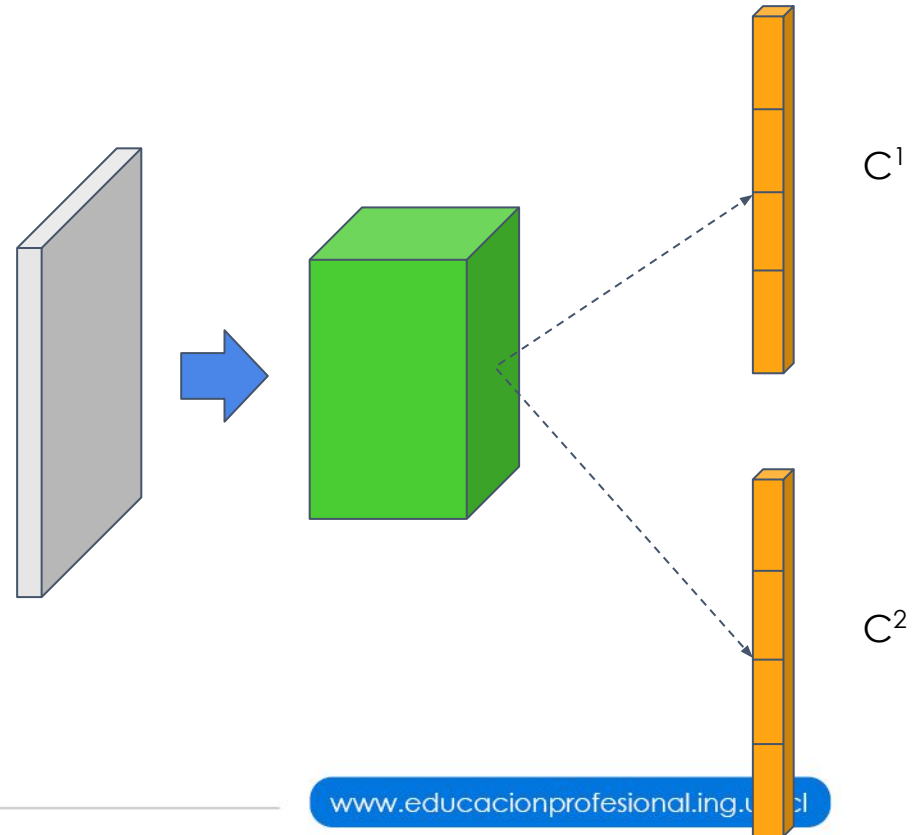
C 2

Task Incremental



El aprendizaje tradicional tiene un clasificador con la cantidad de clases del dataset

Cada Tarea tiene una cantidad de clases determinada





Task Incremental

- Versión más sencilla del problema.
- Sabemos a priori con qué tarea estamos lidiando. O sea tenemos un **identificador de Tarea**.
- Por lo mismo, es un escenario que no es tan frecuente en el mundo real.
- Sin embargo, puede ser que por las particularidades de un problema sea factible.

Class Incremental

- Llegan nuevas clases a nuestro problema en el tiempo.
- Tenemos espacio reservado en nuestro clasificador para esta nueva tarea

Tarea 1



C 1

C 2

Tarea 2



C 3

C 4

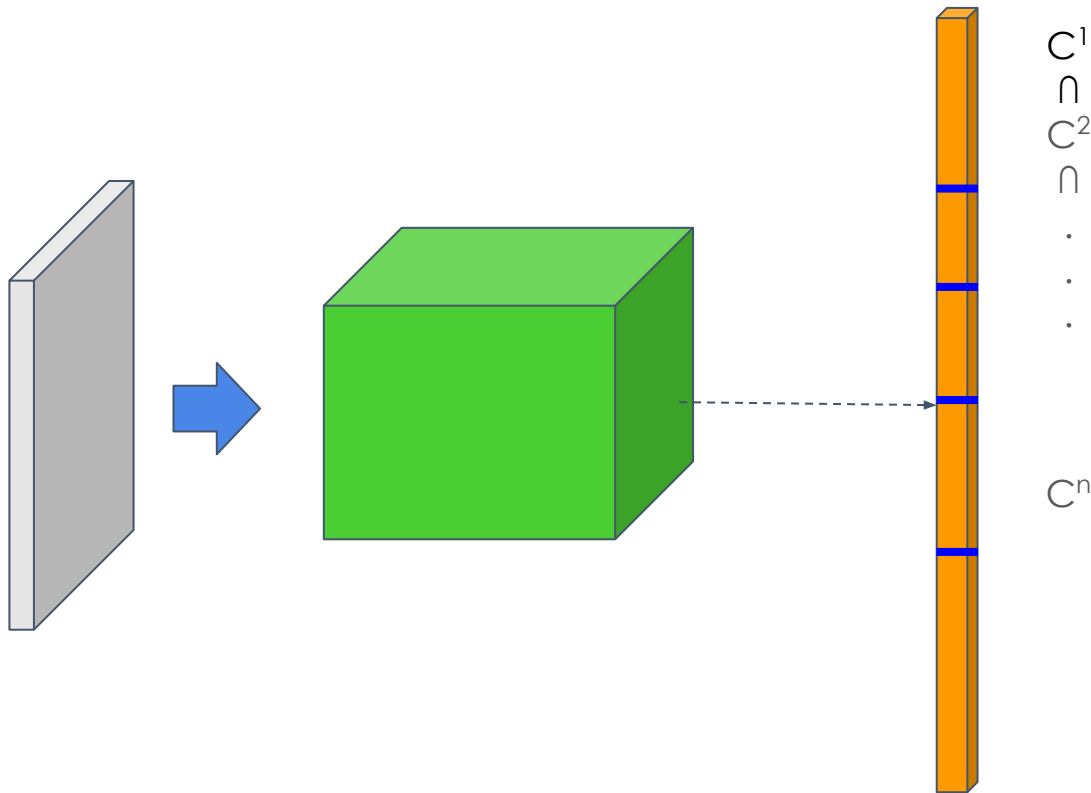
Tarea 3



C 5

C 6

Class Incremental





Class Incremental

- Versión más real del problema
- Como siempre es la misma tarea no necesitamos un identificador de tarea.
- Necesario darse cuenta cuándo llegan elementos nuevos.

Domain Adaptation

- **El dominio del problema cambia en el tiempo**
 - No se agregan nuevas clases
 - ¡Pero sí cambia lo que significan!
- **Dos casos:**
 - Mantener el conocimiento adquirido de la clase en el pasado
 - Olvidar el conocimiento del pasado.



C 1



C 2

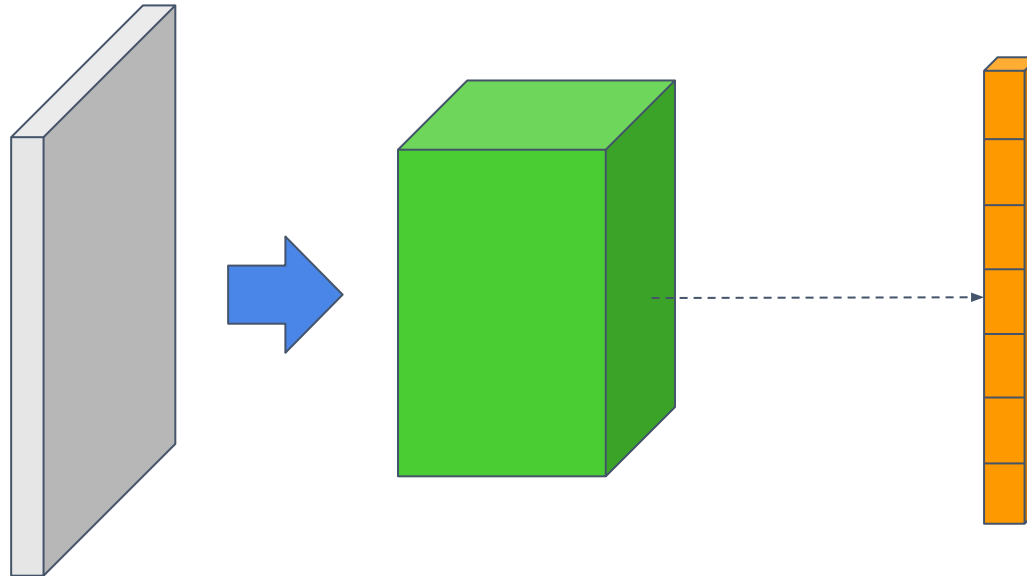


C 1



C 2

Domain Adaptation





Domain Adaptation

- Cambio sutil en la distribución de datos en el tiempo. ¡Hay que darse cuenta!
- No necesitamos agregar nuevos clasificadores ni clases
- Hay que decidir si queremos guardar la representación de tareas pasadas.
 - Ej. Reconocimiento Caras por:
 - Seguridad
 - Facebook



¿Existe una solución al problema?



¿Existe una solución al problema?

¡Por ahora no!



¿Existe una solución al problema?

¡Por ahora no!

- **Es un problema abierto de investigación**
- **Existe soluciones que ayudan a aliviar el problema de Olvido Catastrófico**
 - Pero no son perfectas
 - Depende del problema y contexto



Métricas de Aprendizaje Continuo

- **Mean Accuracy:**

¿Qué tanto estamos aprendiendo entre todas las tareas?

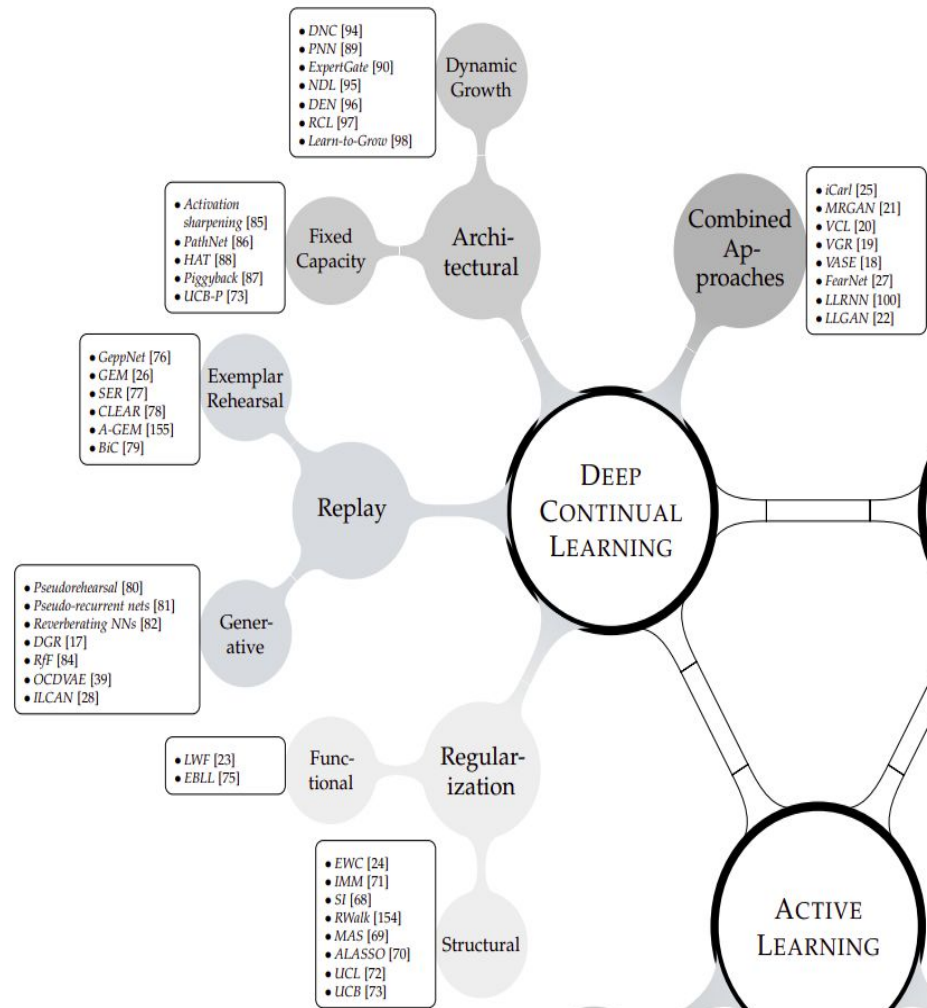
- **Backwards Transfer (BWT)**

¿Qué tanto estamos olvidando?

$$Acc = \frac{1}{T} \sum_{i=1}^T Acc_{T,i}$$

$$BWT = \frac{1}{T-1} \sum_{i=1}^{T-1} Acc_{T,i} - Acc_{i,i}$$

Aprendizaje Continuo es un área en desarrollo





Tipos de Ajustes en Métodos de Aprendizaje Continuo

Regularización

Cambio en la función de pérdida.

Memoria

Utilizamos un subconjunto de ejemplos de tareas pasadas

Arquitectura

Hacemos algún cambio en la arquitectura del modelo.

Elastic Weight Consolidation (EWC)

- **Intuición básica**

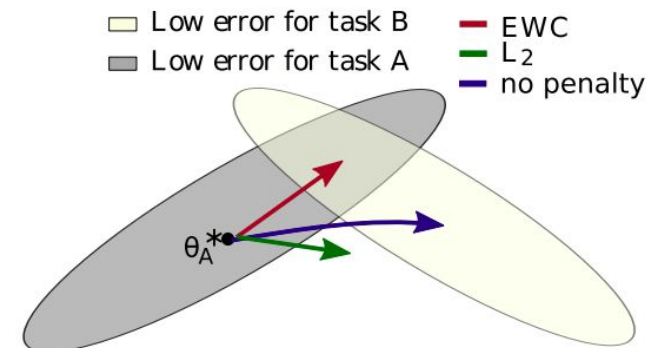
- Limitar interferencia entre tareas, desincentivando que se puedan mover pesos importantes de una tarea al aprender una nueva tarea.

- **Utilizan la matriz de Fisher para determinar la importancia de los pesos para cada tarea**

- Aproximan esta matriz con la primera derivada

$$\mathcal{L}(\theta) = \mathcal{L}_B(\theta) + \sum_i \frac{\lambda}{2} F_i (\theta_i - \theta_{A,i}^*)^2$$

- ☐ Regularización
- ☐ Memoria
- ☐ Arquitectura



Learning Without Forgetting (LwF)

- **Se utilizan funciones de *Distillation***

- Tratamos de hacer que la red se comporte para los datos nuevos como lo hacía la red antes de entrenar con estos datos.
- Obviamente, también lo entrenamos para que aprenda a clasificar los datos nuevos.
- Se entrena el modelo minimizando ambas funciones de pérdida

$$\theta_s^*, \theta_o^*, \theta_n^* \leftarrow \underset{\hat{\theta}_s, \hat{\theta}_o, \hat{\theta}_n}{\operatorname{argmin}} \left(\lambda_o \mathcal{L}_{old}(Y_o, \hat{Y}_o) + \mathcal{L}_{new}(Y_n, \hat{Y}_n) + \mathcal{R}(\hat{\theta}_s, \hat{\theta}_o, \hat{\theta}_n) \right)$$

- ☐ Regularización
- ☐ Memoria
- ☐ Arquitectura

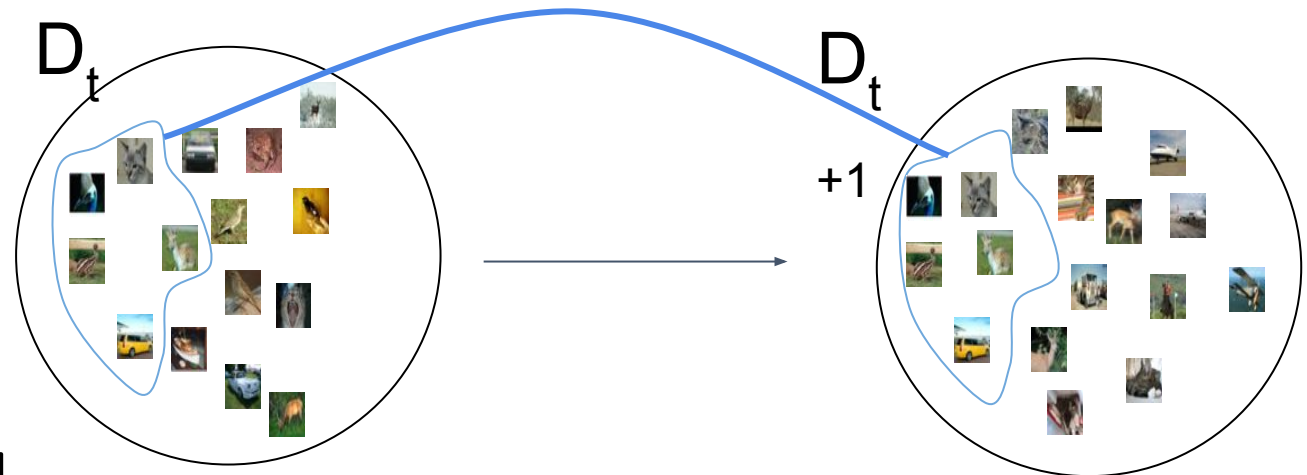


Métodos basados en Memoria

- Guardar **elementos de tareas anteriores**
 - Todos o un subconjunto de los elementos del pasados
 - Viola la idea de no tener acceso a datos de tareas pasadas.
- Éstos se seleccionan de manera *aleatoria, greedy* u otra forma
 - Aleatorio es suficientemente bueno.
- Esta memoria busca ser **representativa** de las tareas anteriores

Experience Replay (ER)

- El método más sencillo basado en memoria.
- Al aprender una nueva tarea, agregamos al batch de entrenamiento los elementos de nuestra memoria.
- Método sencillo y efectivo.



- ☐ Regularización
- ☐ Memoria
- ☐ Arquitectura

Gradient Episodic Memory (GEM)

- Usamos datos en memoria para modificar los gradientes de la nueva tarea.
- Logra una transferencia positiva
 - Si solo un gradiente es bajo, el valor sigue siendo bajo
- Tiene una memoria M_k para cada tarea k
- Se busca que los gradientes de las tareas nuevas no interfieran con los de datos asociados a tareas antiguas

$$\langle g, g_k \rangle := \left\langle \frac{\partial \ell(f_\theta(x, t), y)}{\partial \theta}, \frac{\partial \ell(f_\theta, \mathcal{M}_k)}{\partial \theta} \right\rangle \geq 0, \text{ for all } k < t.$$

- ☐ Regularización
- ☐ Memoria
- ☐ Arquitectura



Métodos basados en Memoria

- **Es necesario guardar datos de tareas anteriores**
 - No siempre factible tener acceso a estos datos
- **Algunos trabajos guardan descriptores/features**
 - Más eficiente y no tiene problemas de seguridad
- **La memoria que utilizan crecen con cada tarea**
 - O tienen un límite de elementos por clase
 - alguna clase puede quedar no representada

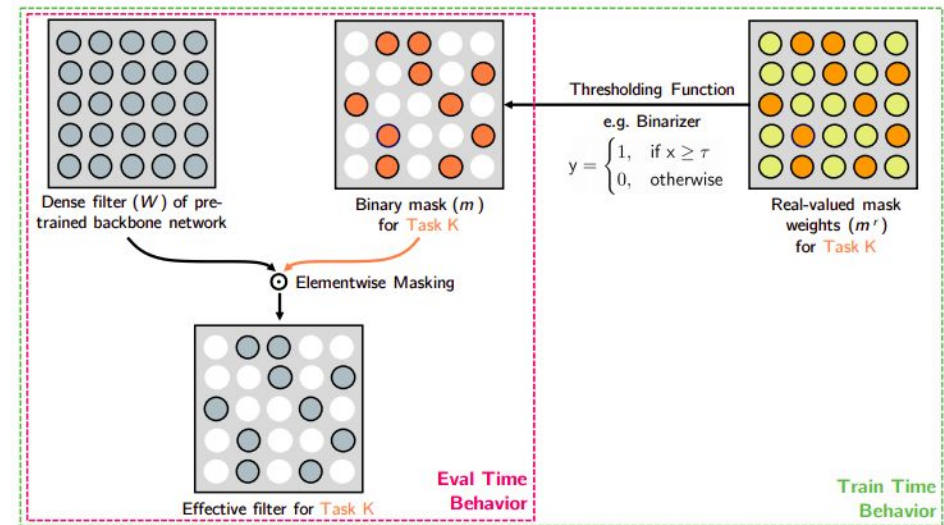


Métodos basados en Arquitectura

- **Son cambios a la arquitectura del modelo**
 - Pueden ser máscaras, funciones de atención, etc.
- **Suelen depender de un identificador de clase**
 - Éste permite alterar la arquitectura de alguna manera para adaptarse a la tarea particular.
- **Se aprovechan de funciones externas para poder limitar el aprendizaje de las nuevas tareas**

PiggyBack

- Asumen una **red pre-entrenada** en un dataset diferente
 - Resnet-18
- Se **entrenan máscaras binarias** que se aplican sobre los **parámetros** de la red para cada tarea.
- En inferencia, usamos las **máscaras binarias sobre la red preentrenada** dependiendo de la tarea.
- Máscara binaria requiere mucho menos espacio que modelo completo.
 - 32 a 64 veces menos espacio requerido.

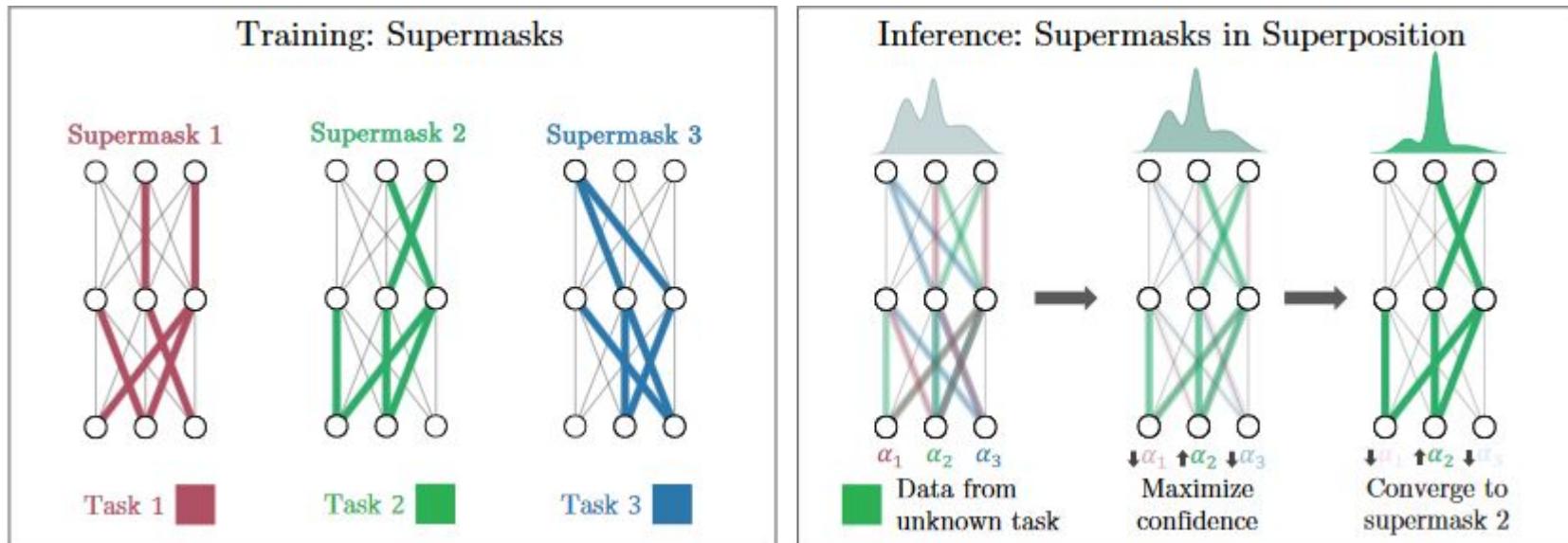


- ☐ Regularización
- ☐ Memoria
- ☐ Arquitectura

SupSup

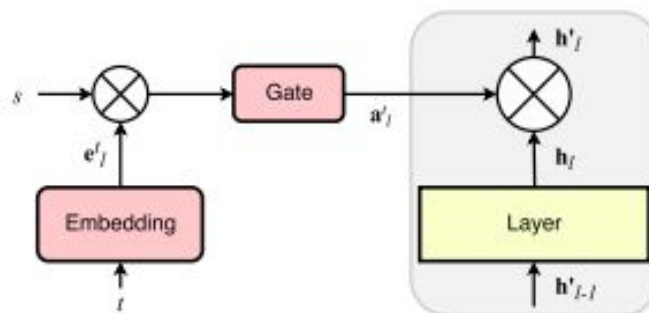
- ☐ Regularización
- ☐ Memoria
- ☐ Arquitectura

- Versión generalizada de Piggyback.
- Puede funcionar en el escenario en que no sabemos ID de tarea.
- Establece un criterio de **incerteza** en la predicción por cada tarea.
- Durante el entrenamiento, este mismo criterio puede ayudar a determinar si estamos cambiando de tarea o no.



Hard Attention to the Task (HAT)

- Utilizan métodos de atención por tarea
- Cada tarea tiene sus propias funciones de atención
 - Determinan la importancia de los pesos



- ☐ Regularización
- ☐ Memoria
- ☐ Arquitectura



Learning to Prompt for Continual Learning (L2P)

- En esencia, este método aprende a consultar un **modelo estático** para resolver nuevas tareas.
- Pensado para funcionar con Transformers.
- Al momento de predecir para una entrada, se determina qué combinación de **N prompts** corresponde para el ejemplo.
- Sólo se entrenan los **prompts** y el **clasificador final**.
- Estos prompts se concatenan como **tokens** al inicio de la secuencia.
 - **Intuición:** le estamos dando instrucciones al modelo estático de qué hacer.

- ☐ Regularización
- ☐ Memoria
- ☐ Arquitectura



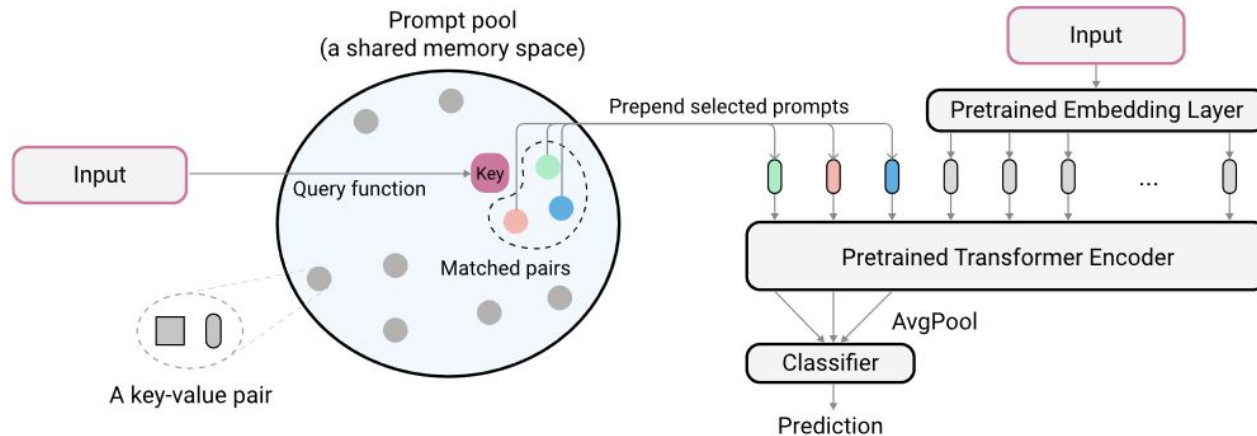
Learning to Prompt for Continual Learning (L2P)

1. Input entra → Via **query - key** se asignan N prompts al input.
 - Opciones para query: usar representación de modelo estático o aprender función.
2. Estos N prompts están asociados a **tokens aprendidos** que se concatenan al inicio del ejemplo.
3. El modelo estático procesa el input con los tokens concatenados.
4. Para predecir, se promedian vectores asociados a tokens de prompts.
5. Ese promedio se pasa por la capa de clasificación final → Output!

- ☐ Regularización
- ☐ Memoria
- ☐ Arquitectura

Learning to Prompt for Continual Learning (L2P)

- Se agrega además una **pérdida** para que las keys sean parecidas a las queries.
 - Queremos que todo input ojalá se parezca a alguna de las llaves para asignarle la tarea correcta.



- ☐ Regularización
- ☐ Memoria
- ☐ Arquitectura

L2P - Resultados

- Reutiliza conocimiento de tareas
- L2P obtiene mejor exactitud promedio que competidores...
- Junto con menor olvido.
- No requiere un task ID

- ☐ Regularización
- ☐ Memoria
- ☐ Arquitectura

Method	Buffer size	Split CIFAR-100	
		Average Acc (↑)	Forgetting (↓)
FT-seq-frozen	0	17.72±0.34	59.09±0.25
FT-seq		33.61±0.85	86.87±0.20
EWC [21]		47.01±0.29	33.27±1.17
LwF [28]		60.69±0.63	27.77±2.17
L2P (ours)		83.83±0.04	7.63±0.30
ER [8]	10/class	67.87±0.57	33.33±1.28
GDumb [46]		67.14±0.37	-
BiC [61]		66.11±1.76	35.24±1.64
DER++ [3]		61.06±0.87	39.87±0.99
Co ² L [4]		72.15±1.32	28.55±1.56
L2P-R (ours)		84.21±0.53	7.72±0.77
ER [8]	50/class	82.53±0.17	16.46±0.25
GDumb [46]		81.67±0.02	-
BiC [61]		81.42±0.85	17.31±1.02
DER++ [3]		83.94±0.34	14.55±0.73
Co ² L [4]		82.49±0.89	17.48±1.80
L2P-R (ours)		86.31±0.59	5.83±0.61
Upper-bound	-	90.85±0.12	-



Resumen de la Clase/Takeaways

- Cuando llegan nuevos datos, no podemos llegar y entrenar sólo en éstos y esperar que todo salga bien.
- Varias consideraciones pueden hacer que **entrenar con todos los datos no sea factible**.
- **Olvido Catastrófico** consiste en perder rendimiento en tareas que el modelo sabía resolver.
- Existen técnicas para paliar esto:
 - Regularizadores
 - Memoria
 - Estructurales
 - ¡Pero no es un problema resuelto!



ESCUELA DE INGENIERÍA
FACULTAD DE INGENIERÍA

EDUCACIÓN
PROFESIONAL



www.educacionprofesional.ing.uc.cl